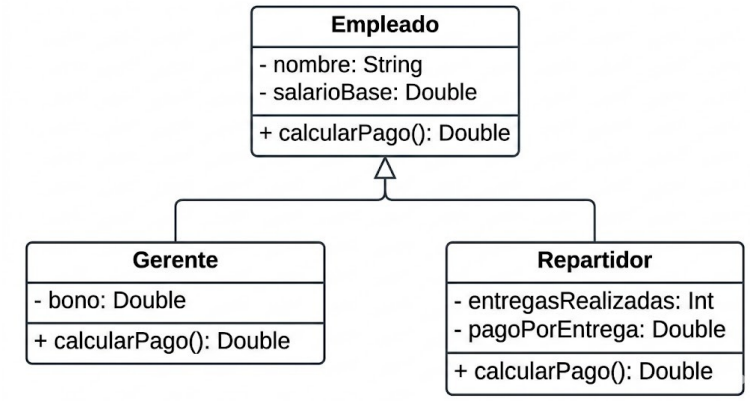
Ejercicios con clases avanzadas

Ejercicio 1A: Sistema de Gestión de Empleados



Este diagrama muestra la clase base **Empleado** y cómo las clases **Gerente** y **Repartidor** heredan de ella, añadiendo sus propios atributos y sobrescribiendo el método de cálculo de pago.

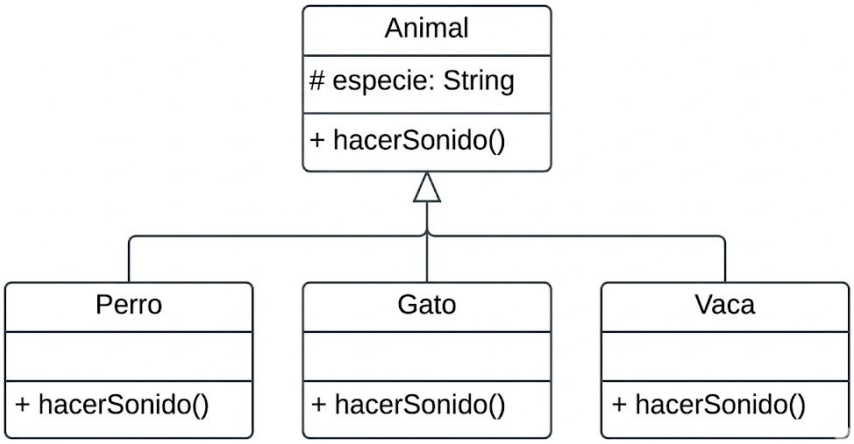
Instrucciones:

1. **Clase Empleado**
   (a) Atributos: nombre y salarioBase.
   (b) Método calcularSueldo(), que simplemente devuelva el salario base
2. **Clase Gerente**:
   (a) Atributo bono.
   (b) Método calcularSueldo(), que debe suamar el salarioBase + bono.
3. **Clase Repartidor**:
   (a) Atributos: entregaRealizadas y pagoPorEntrega.
   (b) Método calcularSueldo(), que añade al salario base una cantidad que se calcula según las entregas realizadas y el pago por entrega.
4. Clase **PruebaEnpleado**: crea un Empleado, un Gerente, un Repartidor y muestra sus sueldos.

Ejercicio 1B: Abstract

Ahora que ya conoces las clases y métodos abstractos, observa el ejercicio 1: parece evidente que la clase Empleado no se va a instanciar ya que lo que se crearían serian instancias especializadas de Gerente o Repartidor. Además, se observa que todas las clase que heredan de Empleado deben tener implementado el método calcularSueldo. Realiza las modificaciones necesarias para llevar a cavo estas acciones.

Ejercicio 2. El Zoológico y el Polimorfismo

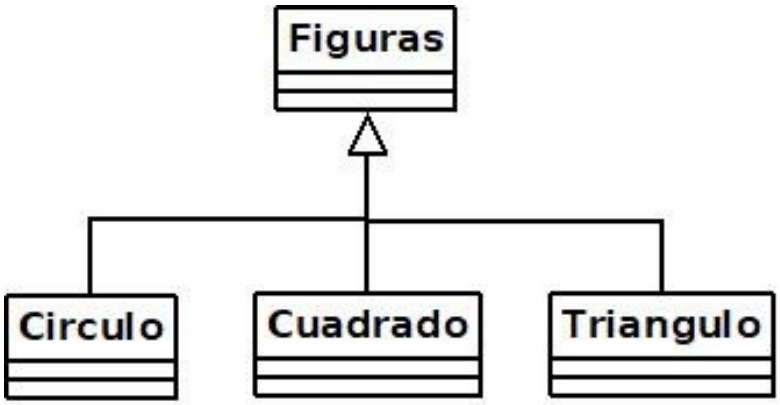


Instrucciones:

1. Crea una clase llamada **Animal.**
    - **Atributo:** especie.
    - **Método:** hacer_sonido() que puede imprimir algo genérico como "El animal hace un ruido".
2. Crea las clases Perro, Gato y Vaca.
    - Cada una debe sobrescribir hacer_sonido() con su sonido característico ("Guau", "Miau", "Muuu").
3. Crea una lista que contenga un ejemplar de cada animal y recorre la lista con un bucle llamando al método hacer_sonido().
4. Intenta declarar la clase Animal como **abstract** para que no se puedan crear animales "genéricos", solo especies concretas.

Ejercicio 3: Figuras geométricas

Implementa una programa con el siguiente esquema de herencia:



Implementa los atributos y métodos necesarios para que se puede ejecutar el siguiente programa:

```
public class Ejercicio1{
    public static void main(String [] args){
        Figuras [] figuras = new Figuras [Figuras.TIPOS];
        figuras[0] = new Circulo(10);
        figuras[1] = new Cuadrado(5);
        figuras[2] = new Triangulo(3,6);
        for(int i=0;i<Figuras.TIPOS;i++)
            System.out.println("Area: " + figuras[i].calcularArea();
    }
}
```

Al ejecutar el programa deberá aparecer por pantalla el área de cada una de las figuras con el siguiente formato:

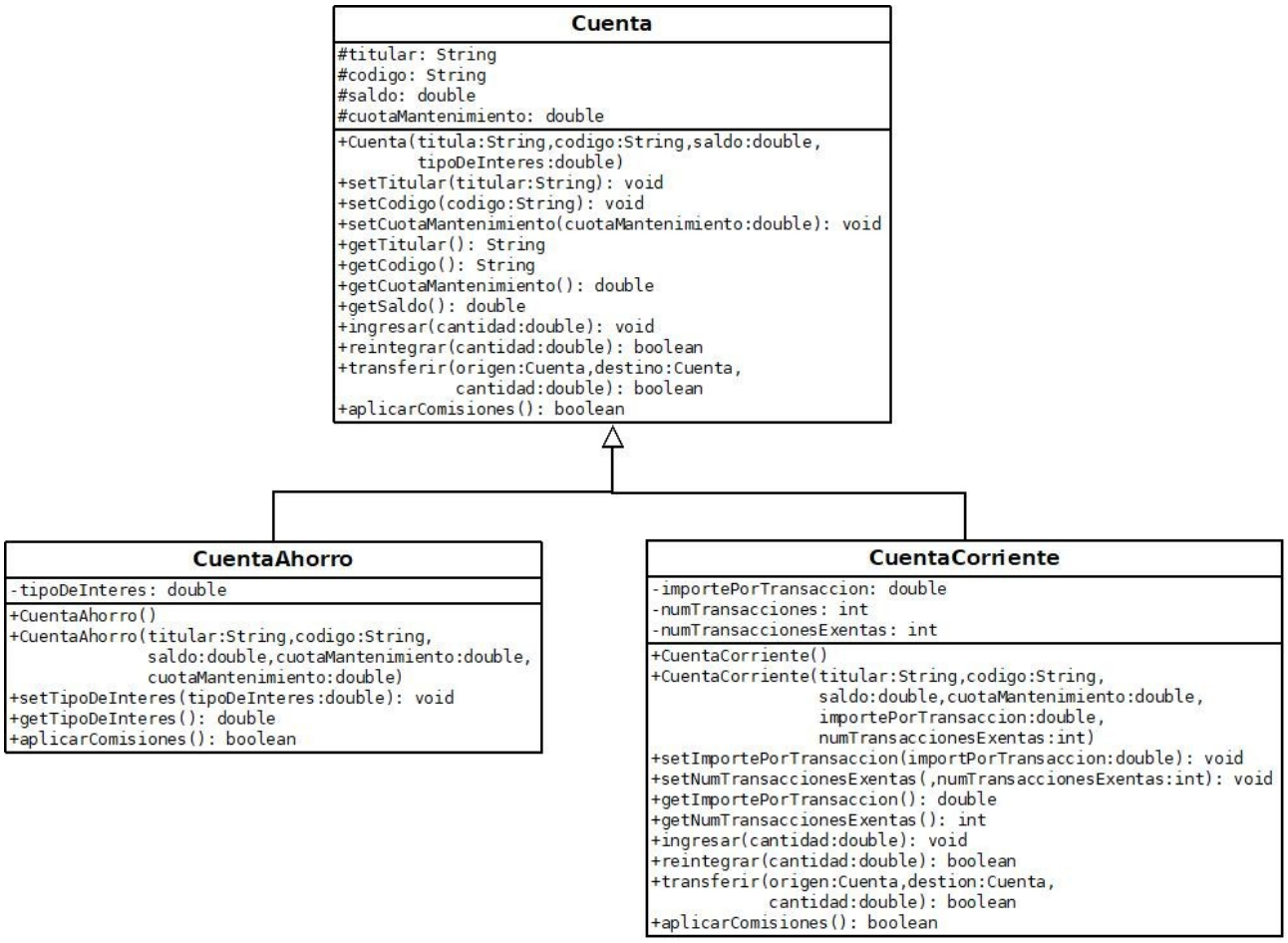
```
Area: 314,16
Area: 25,00
Area: 9,00
```

Nota: observa que solo aparecen dos decimales.

¿Qué has aprendido nuevo con este ejercicio?

Ejercicio 4: Cuentas bancarias

Para manejar las cuentas de un banco se desea implementar el siguiente esquema de herencia:



1) Clase **<u>Cuenta</u>**:
   a) ***aplicarComisiones( )***: le aplica la cuota de mantenimiento.
2) Clase **<u>CuentaAhorro:</u>**
   a) ***aplicarComisiones( )***: le aplica la cuota de mantenimiento y los intereses generados.
3) Clase **<u>CuentaCorriente:</u>**
   a) contabiliza el número de transacciones (ingresos, reintegros y transferencias).
   b) ***aplicarComisiones( ):*** le aplica la cuota de mantenimiento y el importe por transacción por cada transacción que supere el número de transacciones exentas.

Realiza una aplicación para trabajar con estas cuentas:

1. Crea una cuenta de ahorros con el constructor sin parámetros y añadimos cada valor con los métodos adecuados.
2. Crea una cuenta corriente con el constructor con parámetros. Esta cuenta tendrá 1 operación exenta de cobro.
3. Ingresa 10000€ en ambas cuentas y muestra sus saldos.
4. Pasamos 1000€ a la cuenta corriente a la cuenta de ahorro y mostramos los saldos.
5. Llega final del mes y nos aplican las comisiones de ambas cuentas. Muestra los saldos tras aplicar estas comisiones.
6. ¿Has encontrado algún error al implementar las clases siguiendo el diagrama UML?

Ejercicio 5: Vehículos

A) Crea la clase **Vehiculo**, así como las clases **Bicicleta** y **Coche:**

- Para la clase ***Vehículo***, crea los atributos de clase *vehiculosCreados y kmTotales*, así como el atributo de instancia *kmRecorridos*. Añade los siguientes métodos:
  - *getKmRecorridos*
  - *getkmTotales*
  - *getVehiculosCreados*
  - *recorrer*: recibe el número de km que recorre.
- Para la clase ***Bicicleta***, crea el atributo *numPinones y kmParciales.* Añade los siguientes métodos:
  - *getPinones*
  - *getKmParciales*
  - *cambiarPinones:* puede variar el numero de piñones y resetea los kms parciales.
- Para la case ***Coche***, crea el atributo *cilindrada, kmParciales* y *revision=15000*. Añade los siguientes métodos:
  - *getCilindrada*
  - *getKmParciales*
  - *saberKmRevision*: devuelve el número de km que restan para la revisión.
  - revisar: resetea el número de kms parciales.

B) Crea un programa con una bicicleta y un coche. A continuación muestra un menú como el siguientes:

OPCIONES - VEHÍCULOS

==========

1. *Rodar con bicicleta 10 km*
2. *Cambiar piñones – Muestra el cambio realizado*
3. *Ver kilometraje de la bicicleta (parcial/total)*
4. *Ver información completa de la bicicleta*
5. *Rodar con coche 10000 km*
6. *Revisar un coche*
7. *Ver kilometraje del coche (parcial/total)*
8. *Saber lo kilómetros que restan para la revisión del coche*
9. *Ver información completa del coche*
10. *Ver el número de vehículos creados*
11. *Ver kilometraje total de todos los vehículos creados*
12. *Salir*

    Elige una opción (1-12):

==========

C) Añade y modifica los métodos que necesites para realizar todas las opciones del menú.

D) En aquellos métodos que pidas valores al usuario, implementa **control de excepciones**.

- ¿Se podría mejorar el esquema de herencia seguido?

Ejercicio 6: Interfaz

a) Modifica el ejercicio 3 de manera que la la clase Cuadrado implemente la interfaz ***Dibujable.*** Cuando un cuadrado ejecuta el método **dibujar** se podrá dibujar un cuadrado formado por * por pantalla. Este método lanza una excepción si el cuadrado no se puede dibujar.

```
public interface Dibujable {
    public void dibujar() throws Exception;
}
```

b) Dibuja el cuadrado accediendo a su posición en le array de Figuras.:

figuras[1].dibujar()

Ejercicio 7: Biclioteca

Se quiere informatizar una biblioteca. Crea las clases **Publicacion**, **Libro** y **Revista** siguiendo las siguientes restricciones:

- Las clases deben estar implementadas con la jerarquía correcta.
- Las características comunes de las revistas y de los libros son el código ISBN, el título, y el año de publicación.
- Los libros tienen además un atributo prestado para saber si el libro está prestado o no. Cuando se crean los libros, no están prestados.
- Las revistas tienen un número de revista.
- La clase Libro debe implementar la interfaz **Prestable** que tiene los métodos: *prestar*, *devuelver* y *estaPrestado*.

PROGRAMA PRINCIPAL:

```
public class Ejercicio5 {
  public static void main(String[] args) {
    Publicacion [] publicaciones = new Publicacion[3];
    publicaciones[0] = new Libro("123456", "La Ruta Prohibida", 2007);
    publicaciones[1] = new Revista("444555", "Anio Cero", 2019, 344);
    publicaciones[2] = new Publicacion("456789", "La rosa del mundo", 1995);

    for(int i=0;i<3;i++)
      System.out.println(publicaciones[i]);
    System.out.println();
```

```
        //Primer prestamo
        publicaciones[0].prestar();
        if (((Libro)publicaciones[0]).estaPrestado()) {
            System.out.println("El libro esta prestado\n");
        }

        //Segundo prestamo
        publicaciones[1].prestar();
        if (((Libro)publicaciones[1]).estaPrestado()) {
            System.out.println("El libro esta prestado\n");
        }

        //Tercer prestamo
        publicaciones[2]).prestar();
        if (((Libro)publicaciones[2]).estaPrestado()) {
            System.out.println("El libro esta prestado\n");
        }
    }
}
```

Responde:

1. ¿Se usa el polimorfismo en este código? Si es así ¿donde se usa?
2. ¿Por qué fallan los tres prestamos? ¿Se podría solucionar alguno de ellos?